

■ Auto-Director: Interview Preparation Guide

Comprehensive Project Breakdown, File-by-File Analysis & Technical Q&A;

1. Executive Summary (Elevator Pitch)

"Auto-Director is an autonomous, agentic multi-tier AI micro-drama video generator. It converts raw story text (in English, Hindi, or Hinglish) into fully animated, voice-narrated, subtitle-aligned 9:16 vertical short-form videos (YouTube Shorts / Instagram Reels / TikTok).

It features a 5-agent modular pipeline with an audio-driven timing architecture, multi-tier resilience fallbacks (ElevenLabs → gTTS → espeak for voice; Gemini → Pollinations → Stock → PIL for visuals), and custom Devanagari font resolution with 40x faster Pillow-based subtitle rendering."

2. System Architecture & 5-Agent Pipeline

Agent	Primary Provider & Fallbacks	Key Output / Responsibility
1. Scriptwriter	Gemini 2.5 Flash → Offline Regex Splitter	Parses story into 4-8 scenes (narration, visual prompt, mood).
2. Voice	ElevenLabs → gTTS → espeak-ng	Synthesizes parallel audio per scene. Calculates exact audio duration.
3. Visual	Gemini 3.1 Flash → Pollinations → Stock → PIL Cards	Generates 9:16 vertical visual stills with MD5 prompt caching.
4. Subtitles	Proportional Character Split → OpenAI Whisper	Computes word-level start/end timestamps per caption chunk.
5. Director	MoviePy + PIL Engine + FFmpeg H.264 Muxer	Renders Ken Burns zoom, burned-in subtitles, and final MP4 export.

3. Detailed File-by-File Breakdown

File Path	Role & Key Implementation Details
main.py CLI Entry Point & Orchestrator	Built with Typer & Rich. Runs the 5 agents sequentially, measures per-stage latency, handles user interruptions (KeyboardInterrupt), and displays rich terminal progress bars.
config.py Central Pydantic Configuration	Uses pydantic-settings to validate environment variables (.env) at startup. Dynamically computes operating mode (active_mode: LIVE vs DEMO) based on available API keys.
models.py Domain Data Models (DTOs)	Defines Caption and Scene dataclasses. Scene incrementally holds data as it flows through agents. Includes safe dictionary deserialization (Scene.from_dict) without mutating caller data.
agents/scriptwriter_agent.py Agent 1: Scriptwriter	Prompts Gemini for structured JSON scene breakdown. Falls back to deterministic rule-based sentence splitting (_offline_split) if network calls fail.
agents/voice_agent.py Agent 2: Voice Actor	Synthesizes narration audio concurrently via ThreadPoolExecutor. Tries ElevenLabs, then gTTS, then espeak-ng. Measures actual waveform duration from .wav file (critical for downstream timing).
agents/visual_agent.py Agent 3: Cinematographer	Generates vertical 9:16 images concurrently. Employs 4-tier fallback: Gemini Image Gen -> Pollinations Turbo -> Stock Photos -> PIL Mood Gradients. Uses MD5 hash caching for prompts.
agents/subtitle_agent.py Agent 4: Subtitle Sync Engineer	Calculates 3-word caption timings. Uses character-length proportional timing by default, or OpenAI Whisper forced alignment (USE_WHISPER=true) for real word-level timestamps.
agents/director_agent.py Agent 5: Video Renderer	Applies Ken Burns scaling zoom (_ken_burns_clip). Renders subtitle overlays with dynamic Devanagari/Latin font resolution (_resolve_font) using Pillow. Guarantees resource cleanup with try...finally clip closing.
core/retry.py Resilience Decorator	Custom @retry_api_call decorator using tenacity for exponential backoff. Intelligent exception predicate filters out non-transient HTTP 4xx errors (e.g. 401 Unauthorized, 402 Payment Required).
core/exceptions.py Typed Error Hierarchy	Custom pipeline exception tree inheriting from AutoDirectorError (e.g. ScriptwriterError, VoiceError, VisualError, DirectorError) for explicit stage-specific error handling.
core/logger.py Rich Terminal Logger	Configures unified logging output across all modules with colorized levels.

4. Data Structures, Algorithms & CS Concepts Used

Concept / Category	Implementation & Usage in Code
Typed State Machine (DTOs)	Custom Scene & Caption dataclasses (models.py). Validation helpers (is_voice_ready, is_render_ready) enforce clean state transitions across agents.
Multithreading & Concurrency	ThreadPoolExecutor in voice_agent.py and visual_agent.py processes scene tasks concurrently (max_workers=4), accelerating execution by ~6x.
Proportional Timing Algorithm	Math-based linear duration allocation in subtitle_agent.py: $T_{chunk} = T_{scene} * (Chars_{chunk} / Chars_{total})$ when Whisper is unavailable.
Mathematical Animation Scale	Time-dependent continuous zoom function in director_agent.py: $S(t) = S_{base} * (1 + k * t/T)$ for smooth Ken Burns pan/zoom motion.
Prompt Memoization (Caching)	Hash-map disk caching with MD5 hashes of visual prompts (visual_agent.py) to bypass duplicate AI image generation API calls.
Exponential Backoff Retries	Decorator pattern in core/retry.py using Tenacity. Retries transient 429/5xx status codes while skipping non-retryable 4xx errors.
Unicode Script Inspection	O(N) character range inspection (0x0900-0x097F) in director_agent.py for dynamic font path assignment (Devanagari vs Latin).

5. Key Engineering Highlights & Architectural Decisions

- **Audio-Driven Timing Architecture:** Scene lengths are strictly driven by the actual waveform duration of the synthesized audio (AudioFileClip.duration). Subtitles and Ken Burns animation timings are bound to this value for 100% lip/sound sync.
- **Multi-Tier Fallback Resilience:** Zero hard failure points. If external APIs fail or rate limit, the system gracefully degrades down the provider chain (e.g. ElevenLabs -> gTTS -> espeak-ng) without crashing.
- **Pillow-Based Subtitle Engine:** Avoids ImageMagick dependency bottlenecks by pre-rendering captions to transparent PNGs with PIL (stroke_width=3, stroke_fill=black, rounded pill background). Yields 40x faster rendering speed.
- **Multilingual Font Resolution:** Inspects text for Unicode Devanagari range (0x0900 - 0x097F) to select NotoSansDevanagari vs. DejaVuSans, eliminating font box ('tofu') rendering bugs.
- **Strict Clip Lifecycle Cleanup:** MoviePy clips are managed inside try...finally blocks, ensuring clip.close() is called on all video/audio handles to prevent FFmpeg process leaks.

6. Top Technical Interview Q&A; Cheatsheet

Q1: How do you ensure audio and video remain perfectly synchronized?

Answer: We use an Audio-Driven Timing Architecture. We never assume a fixed scene duration. VoiceAgent synthesizes audio first and extracts the exact duration from the audio waveform. All video clip lengths, Ken Burns scaling progress, and caption timestamp chunks are calculated dynamically based on this exact duration.

Q2: How does the system handle external API rate limits or outages?

Answer: Each agent encapsulates a multi-tier fallback mechanism wrapped with exponential backoff retries via Tenacity. If a primary service like ElevenLabs or Gemini fails or returns 429, the system logs a warning and gracefully falls back to local/free alternatives like gTTS, espeak-ng, Pollinations Turbo, or PIL gradient cards.

Q3: Why did you build custom PIL subtitle rendering instead of using MoviePy's TextClip?

Answer: MoviePy's default TextClip relies on ImageMagick binaries, which are slow, platform-dependent, and struggle with Hindi Devanagari font wrapping. By rendering captions onto transparent PNGs in memory using Pillow, we achieved 40x faster rendering, perfect rounded pill contrast overlays, and cross-platform reliability.

Q4: How do you support multilingual stories (English vs. Hindi)?

Answer: We implement dynamic font resolution in DirectorAgent. It scans subtitle text for Devanagari Unicode characters (0x0900-0x097F). If found, it automatically assigns NotoSansDevanagari.ttf; otherwise, it uses DejaVuSans-Bold.ttf. Additionally, ScriptwriterAgent ensures image prompts are always generated in English for AI models while narration matches the input language.

Q5: How do you prevent resource and process leaks during video rendering?

Answer: Video compositing spawns multiple underlying FFmpeg subprocesses. We encapsulate the entire compositing loop in DirectorAgent with a strict try...finally block that explicitly calls .close() on all image, audio, and composite clips even if rendering fails midway.